

Resumen de Fundamentos de Programación en C++

Arreglos, Búsquedas, Ordenamientos y Funciones

1. Arreglos (Estructuras de Datos)

Arreglos Unidimensionales

Un arreglo es como una fila de cajas en la memoria, donde cada caja puede guardar un valor del mismo tipo [cite: 4]. Cada elemento se identifica con un índice que empieza desde 0 [cite: 4]. Su tamaño es fijo y no se puede cambiar en tiempo de ejecución [cite: 4].

Para recorrerlos, se suele utilizar un ciclo `for` que va desde la posición 0 hasta la última [cite: 4].

```
#include <iostream>
using namespace std;

int main() {
    // Declaración de arreglo unidimensional
    int numeros[5] = {10, 20, 30, 40, 50};

    // Recorrido con ciclo for
    for (int i = 0; i < 5; i++) {
        cout << "Posición " << i << ": " << numeros[i] << endl;
    }
    return 0;
}
```

Arreglos Bidimensionales

Un arreglo bidimensional es como una tabla o cuadrícula con filas y columnas [cite: 1]. Cada dato se accede con dos posiciones: fila y columna (ejemplo: `matriz[1][2]`) [cite: 1].

Se necesitan dos ciclos anidados para recorrerlos: un ciclo externo para las filas y uno interno para las columnas [cite: 1].

```
int tabla[2][3] = {
    {1, 2, 3}, // Fila 0
    {4, 5, 6}  // Fila 1
};

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        cout << "Elemento [" << i << "][" << j << "] = " << tabla[i][j] <<
endl;
    }
}
```

Arreglos Dinámicos

A diferencia de los arreglos estáticos (memoria Stack), los dinámicos permiten definir su tamaño durante la ejecución del programa [cite: 3].

- Utilizan memoria dinámica conocida como **Heap** [cite: 3].
- Se requiere el uso de punteros para guardar la dirección de memoria [cite: 3].
- Se usa el operador `new` para reservar la memoria y `delete` para liberarla [cite: 3].

Es crucial liberar la memoria con `delete` para evitar fugas de memoria [cite: 3].

```
int n = 3;
// Reserva de memoria en el Heap
int* p = new int[n];

p[0] = 5; p[1] = 10; p[2] = 15;

// Liberar la memoria al terminar
delete[] p;
```

2. Algoritmos de Búsqueda

Búsqueda Lineal (Secuencial)

Consiste en recorrer el arreglo desde el primer elemento hasta el último, comprobando uno por uno si coincide con el valor buscado [cite: 6].

- **Ventajas:** Es fácil de implementar y no requiere que los datos estén ordenados [cite: 6].
- **Desventajas:** Puede ser muy lenta en arreglos grandes [cite: 6].

```
int arreglo[] = {4, 7, 2, 9, 1};
int valor = 9;
bool encontrado = false;

for (int i = 0; i < 5; i++) {
    if (arreglo[i] == valor) {
        encontrado = true;
        cout << "Encontrado en posición " << i << endl;
        break; // Detiene la búsqueda
    }
}
```

Búsqueda Binaria

Consiste en examinar el elemento central del arreglo; si el valor buscado es menor, se busca en la mitad inferior, si es mayor, en la mitad superior [cite: 10].

- **Condición obligatoria:** El arreglo debe estar previamente ordenado [cite: 10].
- **Ventajas:** Es mucho más rápida (complejidad logarítmica) [cite: 10].

```

int arreglo[] = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67}; // Ordenado
int inicio = 0, fin = 9, valor = 16, medio;
bool encontrado = false;

while (inicio <= fin) {
    medio = (inicio + fin) / 2;
    if (arreglo[medio] == valor) {
        encontrado = true;
        break;
    } else if (valor < arreglo[medio]) {
        fin = medio - 1; // Mitad izquierda
    } else {
        inicio = medio + 1; // Mitad derecha
    }
}

```

3. Algoritmos de Ordenamiento

El ordenamiento organiza los datos de forma lógica, lo cual facilita búsquedas y toma de decisiones [cite: 8].

La función sort()

En C++, la función `sort()` pertenece a la librería estándar STL (`<algorithm>`) y utiliza algoritmos avanzados (como IntroSort) para ordenar rápidamente [cite: 8].

```

#include <algorithm>
// ...
int numeros[] = {7, 2, 5, 1, 9};
int tam = sizeof(numeros) / sizeof(numeros[0]); // Calcula el tamaño
sort(numeros, numeros + tam); // Ordena de menor a mayor

```

Ordenamiento Burbuja (Bubble Sort)

Compara elementos de dos en dos (pares consecutivos) y los intercambia si están desordenados, repitiendo el proceso hasta que el mayor "flote" al final [cite: 10].

```
for (int i = 0; i < tamano - 1; i++) {  
    for (int j = 0; j < tamano - 1; j++) {  
        if (arreglo[j] > arreglo[j + 1]) {  
            int temp = arreglo[j];  
            arreglo[j] = arreglo[j + 1];  
            arreglo[j + 1] = temp;  
        }  
    }  
}
```

4. Funciones

Una función es un bloque de código que realiza una tarea específica, permitiendo dividir el programa en partes más pequeñas y reutilizables [cite: 5]. Mejoran la legibilidad y mantenibilidad del código [cite: 5].

Tipos de Funciones

Las funciones se clasifican según si reciben parámetros y si retornan un valor [cite: 2, 7]:

1. **Sin parámetros y sin retorno:** Ejecutan una acción general (usan `void`) [cite: 2].
2. **Con parámetros y sin retorno:** Reciben datos para trabajar pero no devuelven resultado (usan `void`) [cite: 2].
3. **Sin parámetros y con retorno:** Entregan un resultado (usan `return` y un tipo de dato como `int`) [cite: 2].
4. **Con parámetros y con retorno:** Reciben datos y devuelven un cálculo (usan `return`) [cite: 2].

```
// 1. Sin parámetros y sin retorno
void mostrarMenu() {
    cout << "Menú Principal" << endl;
}

// 2. Con parámetros y sin retorno
void saludar(string nombre) {
    cout << "Hola " << nombre << endl;
}

// 4. Con parámetros y con retorno
int sumar(int a, int b) {
    return a + b;
}
```